torch.einsum#

https://pytorch.org/docs/stable/generated/torch.einsum.html#torch.einsum

Description:

Sums the product of the elements of the input operands along dimensions specified using a notation based on the Einstein summation convention. Einsum allows computing many common multi-dimensional linear algebraic array operations by representing them in a short-hand format based on the Einstein summation convention, given by equation. The details of this format are described below, but the general idea is to label every dimension of the input operands with some subscript and define which subscripts are part of the output. The output is then computed by summing the product of the elements of the operands along the dimensions whose subscripts are not part of the output. For example, matrix multiplication can be computed using einsum as torch.einsum(“ij,jk->ik”, A, B). Here, j is the summation subscript and i and k the output subscripts (see section below for more details on why). The equation string specifies the subscripts (letters in [a-zA-Z]) for each dimension of the input operands in the same order as the dimensions, separating subscripts for each operand by a comma (‘,’), e.g. ‘ij,jk’ specify subscripts for two 2D operands. The dimensions labeled with the same subscript must

Function Signature

```
torch.einsum(equation, *operands) → Tensor[source]#
Parameters
Return type
```

Returns:

Tensor

Code Examples

```
>>> # trace
>>> torch.einsum('ii', torch.randn(4, 4))
tensor(-1.2104)

>>> # diagonal
>>> torch.einsum('ii->i', torch.randn(4, 4))
tensor([-0.1034,  0.7952, -0.2433,  0.4545])

>>> # outer product
>>> x = torch.randn(5)
>>> y = torch.randn(4)
>>> torch.einsum('i,j->ij', x, y)
tensor([[ 0.1156, -0.2897, -0.3918,  0.4963],
        [-0.3744,  0.9381,  1.2685, -1.6070],
        [ 0.7208, -1.8058, -2.4419,  3.0936],
```

```
        [ 0.1713, -0.4291, -0.5802,  0.7350],
        [ 0.5704, -1.4290, -1.9323,  2.4480]])

>>> # batch matrix multiplication
>>> As = torch.randn(3, 2, 5)
>>> Bs = torch.randn(3, 5, 4)
>>> torch.einsum('bij,bjk->bik', As, Bs)
tensor([[[-1.0564, -1.5904,  3.2023,  3.1271],
        [-1.6706, -0.8097, -0.8025, -2.1183]],

        [[ 4.2239,  0.3107, -0.5756, -0.2354],
        [-1.4558, -0.3460,  1.5087, -0.8530]],

        [[ 2.8153,  1.8787, -4.3839, -1.2112],
        [ 0.3728, -2.1131,  0.0921,  0.8305]]])

>>> # with sublist format and ellipsis
>>> torch.einsum(As, [..., 0, 1], Bs, [..., 1, 2], [..., 0, 2])
tensor([[[-1.0564, -1.5904,  3.2023,  3.1271],
        [-1.6706, -0.8097, -0.8025, -2.1183]],

        [[ 4.2239,  0.3107, -0.5756, -0.2354],
        [-1.4558, -0.3460,  1.5087, -0.8530]],

        [[ 2.8153,  1.8787, -4.3839, -1.2112],
        [ 0.3728, -2.1131,  0.0921,  0.8305]]])

>>> # batch permute
>>> A = torch.randn(2, 3, 4, 5)
>>> torch.einsum('...ij->...ji', A).shape
torch.Size([2, 3, 5, 4])

>>> # equivalent to torch.nn.functional.bilinear
>>> A = torch.randn(3, 5, 4)
>>> l = torch.randn(2, 5)
>>> r = torch.randn(2, 4)
>>> torch.einsum('bn,anm,bm->ba', l, A, r)
tensor([[-0.3430, -5.2405,  0.4494],
```

```
        [ 0.3311,  5.5201, -3.0356]])
```

Notes & Warnings

NOTE Note torch.einsum handles ellipsis (‘…’) differently from NumPy in that it allows dimensions covered by the ellipsis to be summed over, that is, ellipsis are not required to be part of the output.

NOTE Note Please install opt-einsum (https://optimized-einsum.readthedocs.io/en/stable/) in order to enroll into a more performant einsum. You can install when installing torch like so: pip install torch[opt-einsum] or by itself with pip install opt-einsum. If opt-einsum is available, this function will automatically speed up computation and/or consume less memory by optimizing contraction order through our opt_einsum backend torch.backends.opt_einsum (The _ vs - is confusing, I know). This optimization occurs when there are at least three inputs, since the order does not matter otherwise. Note that finding the optimal path is an NP-hard problem, thus, opt-einsum relies on different heuristics to achieve near-optimal results. If opt-einsum is not available, the default order is to contract from

NOTE

Note As of PyTorch 1.10 torch.einsum() also supports the sublist format (see examples below). In this format, subscripts for each operand are specified by sublists, list of integers in the range [0, 52). These sublists follow their operands, and an extra sublist can appear at the end of the input to specify the output's subscripts., e.g. torch.einsum(op1, sublist1, op2, sublist2, …, [subslist_out]). Python's Ellipsis object may be provided in a sublist to enable broadcasting as described in the Equation section above.

Detailed Documentation

Documentation

Sums the product of the elements of the input operands along dimensions specified using a notation based on the Einstein summation convention.

Einsum allows computing many common multi-dimensional linear algebraic array operations by representing them in a short-hand format based on the Einstein summation convention, given by equation. The details of this format are described below, but the general idea is to label every dimension of the input operands with some subscript and define which subscripts are part of the output. The output is then computed by summing the product of the elements of the operands along the dimensions whose subscripts are not part of the output. For example, matrix multiplication can be computed using einsum as torch.einsum("ij,jk->ik", A, B). Here, j is the summation subscript and i and k the output subscripts (see section below for more details on why).

The equation string specifies the subscripts (letters in [a-zA-Z]) for each dimension of the input operands in the same order as the dimensions, separating subscripts for each operand by a comma (‘,’), e.g. ‘ij,jk’ specify subscripts for two 2D operands. The dimensions labeled with the same subscript must be broadcastable, that is, their size must either match or be 1. The exception is if a subscript is repeated for the same input operand, in which case the dimensions labeled with this subscript for this operand must match in size and the operand will be replaced by its diagonal along these dimensions. The subscripts that appear exactly once in the equation will be part of the output, sorted in increasing alphabetical order. The output is computed by multiplying the input operands element-wise, with their dimensions aligned based on the subscripts, and then summing out the dimensions whose subscripts are not part of the output.

Optionally, the output subscripts can be explicitly defined by adding an arrow (‘->’) at the end of the equation followed by the subscripts for the output. For instance, the following equation computes the transpose of a matrix multiplication: ‘ij,jk->ki’. The output subscripts must appear at least once for some input operand and at most once for the output.

Ellipsis (‘…’) can be used in place of subscripts to broadcast the dimensions covered by the ellipsis. Each input operand may contain at most one ellipsis which will cover the dimensions not covered by subscripts, e.g. for an input operand with 5 dimensions, the ellipsis in the equation ‘ab…c’ cover the third and fourth dimensions. The ellipsis does not need to cover the same number of dimensions across the operands but the ‘shape’ of the ellipsis (the size of the dimensions covered by them) must broadcast together. If the output is not explicitly defined with the arrow (‘->’) notation, the ellipsis will come first in the output (left-most dimensions), before the subscript labels that appear exactly once for the input operands. e.g. the following equation implements batch matrix multiplication ‘…ij,…jk’.

A few final notes: the equation may contain whitespaces between the different

elements (subscripts, ellipsis, arrow and comma) but something like '…' is not valid. An empty string '' is valid for scalar operands.

torch.einsum handles ellipsis ('…') differently from NumPy in that it allows dimensions covered by the ellipsis to be summed over, that is, ellipsis are not required to be part of the output.

Please install opt-einsum (https://optimized-einsum.readthedocs.io/en/stable/) in order to enroll into a more performant einsum. You can install when installing torch like so: pip install torch[opt-einsum] or by itself with pip install opt-einsum.

If opt-einsum is available, this function will automatically speed up computation and/or consume less memory by optimizing contraction order through our opt_einsum backend torch.backends.opt_einsum (The _ vs - is confusing, I know). This optimization occurs when there are at least three inputs, since the order does not matter otherwise. Note that finding the optimal path is an NP-hard problem, thus, opt-einsum relies on different heuristics to achieve near-optimal results. If opt-einsum is not available, the default order is to contract from left to right.

To bypass this default behavior, add the following to disable opt_einsum and skip path calculation: torch.backends.opt_einsum.enabled = False

To specify which strategy you'd like for opt_einsum to compute the contraction path, add the following line: torch.backends.opt_einsum.strategy = 'auto'. The default strategy is 'auto', and we also support 'greedy' and 'optimal'. Disclaimer that the runtime of 'optimal' is factorial in the number of inputs! See more details in the opt_einsum documentation (https://optimized-einsum.readthedocs.io/en/stable/path_finding.html).

As of PyTorch 1.10 torch.einsum() also supports the sublist format (see examples below). In this format, subscripts for each operand are specified by sublists, list of

integers in the range [0, 52). These sublists follow their operands, and an extra sublist can appear at the end of the input to specify the output's subscripts., e.g. torch.einsum(op1, sublist1, op2, sublist2, …, [subslist_out]). Python's Ellipsis object may be provided in a sublist to enable broadcasting as described in the Equation section above.

equation (str) – The subscripts for the Einstein summation.

operands (List[Tensor]) – The tensors to compute the Einstein summation of.
